



MobilityDB: Bringing Mobility Data Inside PostgreSQL

PostgreSQL Meetup Belgium

Brussels, March 10, 2026

Contact: Esteban Zimányi (esteban.zimanyi@ulb.be)
Mahmoud SAKR (mahmoud.sakr@ulb.be)



MobilityDB

- A moving object database (MOD)
- Builds on PostgreSQL and PostGIS
- Developed by a team in Université libre de Bruxelles
- [Open source](#) extension
- Compliant with OGC standards on Moving Features, and in particular the OGC Moving Features Access



Types of Mobility Data: Time-Varying Attributes



Position: tgeompoint, tgeogpoint

Speed: tfloat

Speed > 50: tbool

Gear: tint

Road type (e.g., 'highway', 'primary' ...): ttext

Connected Car Data

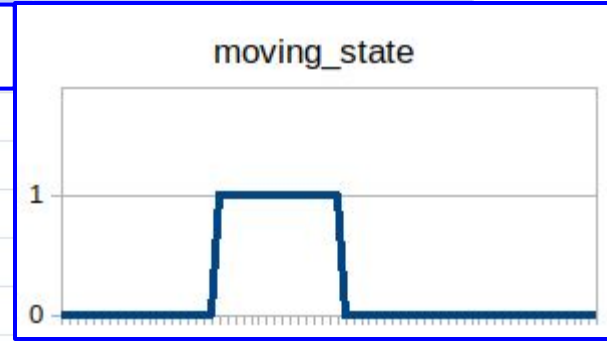
timestamp	moving_state	trip_id	sp1	ssa	driver_gender	driver_age	speed_label
10:10.1		1 c521b0b0	13.64	309	F	32	city
10:10.2		1 c521b0b0	14.2	288	F	32	city
10:10.3		1 c521b0b0	14.56	256.5	F	32	city
10:10.4		1 c521b0b0	14.87		F	32	city
10:10.5		1 c521b0b0	15.19		F	32	city
10:10.6		1 c521b0b0	15.68		F	32	city
10:10.7		1 c521b0b0	16.15		F	32	city
10:10.8		1 c521b0b0	16.7	93	F	32	city
10:10.9		1 c521b0b0	16.99	69	F	32	city
10:11.0		1 c521b0b0	17.55	49.5	F	32	city
10:11.1		1 c521b0b0	18.14		F	32	city
10:11.2		1 c521b0b0	18.7		F	32	city
10:11.3		1 c521b0b0	18.86		F	32	city
10:11.4		1 c521b0b0	19.58		F	32	city
10:11.5		1 c521b0b0	20.21	-25.5	F	32	city
10:11.6		1 c521b0b0	20.63	-36	F	32	city
10:11.7		1 c521b0b0	21.02	-37.5	F	32	city

Connected Car Data

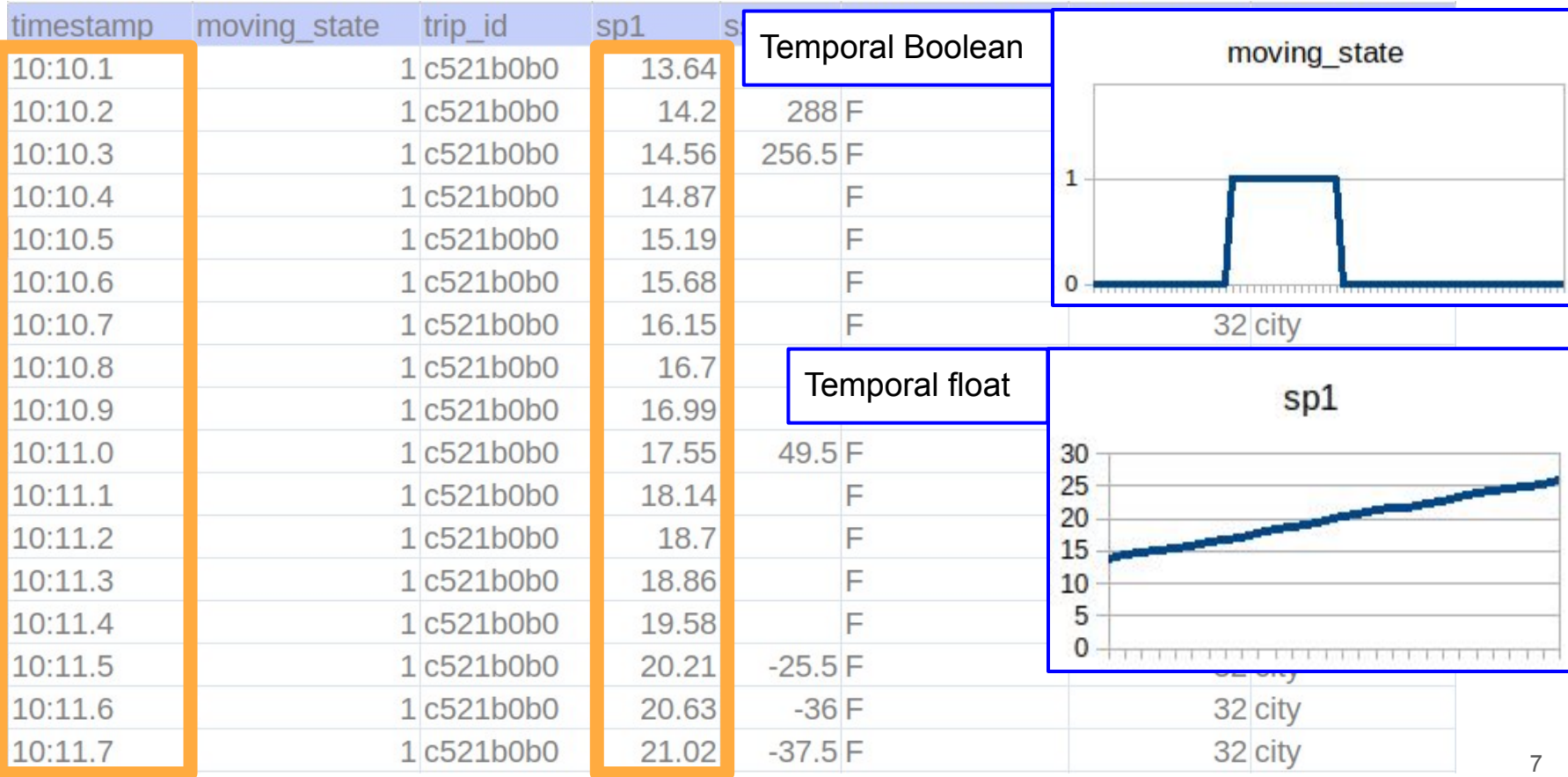
timestamp	moving	state	trip_id	sp1	ssa	driver_gender	driver_age	speed_label
10:10.1		1	e521b0b0	13.64	309	F	32	city
10:10.2		1	e521b0b0	14.2	288	F	32	city
10:10.3		1	e521b0b0	14.56	256.5	F	32	city
10:10.4		1	e521b0b0	14.87		F	32	city
10:10.5		1	e521b0b0	15.19		F	32	city
10:10.6		1	e521b0b0	15.68		F	32	city
10:10.7		1	e521b0b0	16.15		F	32	city
10:10.8		1	e521b0b0	16.7	93	F	32	city
10:10.9		1	e521b0b0	16.99	69	F	32	city
10:11.0		1	e521b0b0	17.55	49.5	F	32	city
10:11.1		1	e521b0b0	18.14		F	32	city
10:11.2		1	e521b0b0	18.7		F	32	city
10:11.3		1	e521b0b0	18.86		F	32	city
10:11.4		1	e521b0b0	19.58		F	32	city
10:11.5		1	e521b0b0	20.21	-25.5	F	32	city
10:11.6		1	e521b0b0	20.63	-36	F	32	city
10:11.7		1	e521b0b0	21.02	-37.5	F	32	city

Connected Car Data

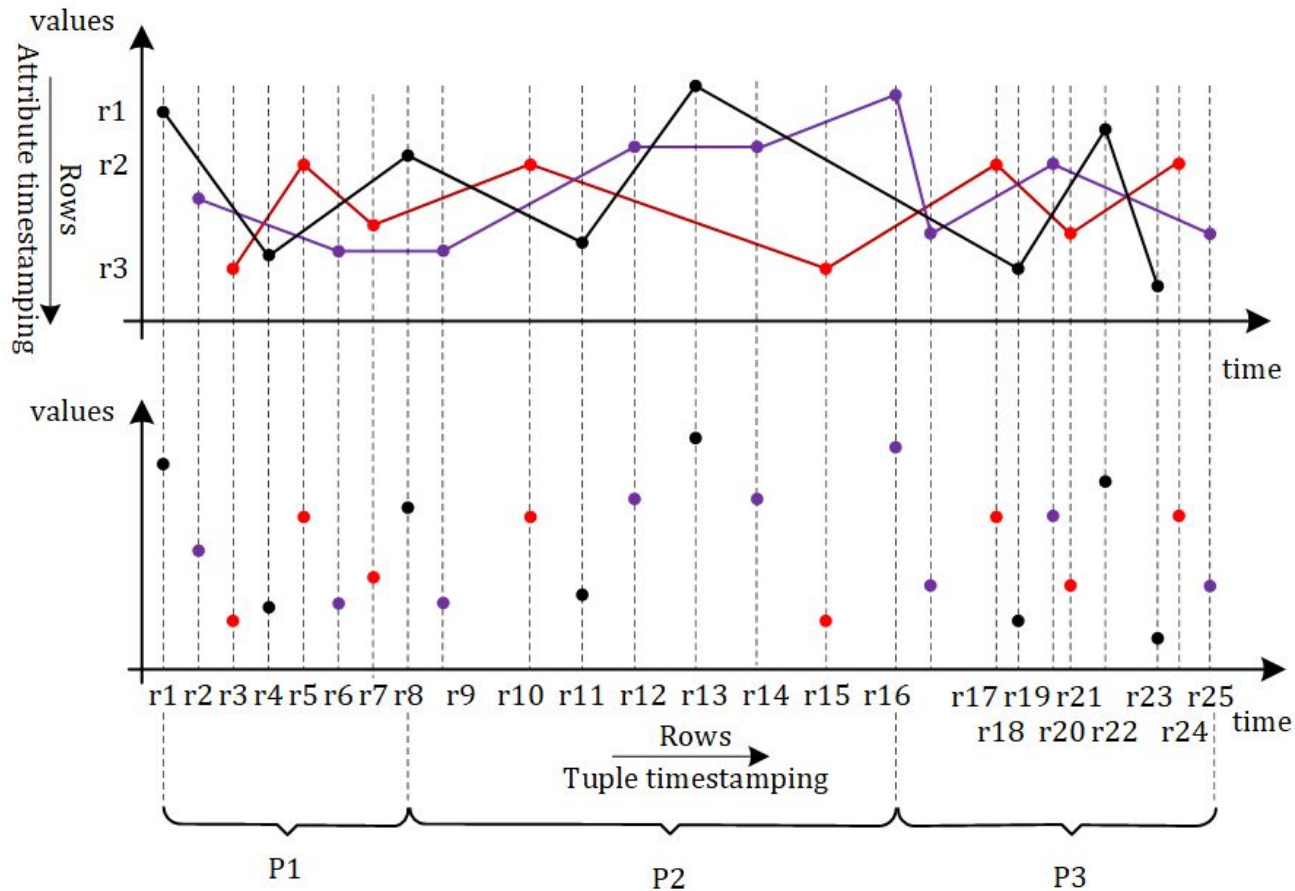
timestamp	moving	state	trip_id	sp1	s	Temporal Boolean		moving_state	
10:10.1			1e521b0b0	13.64					
10:10.2			1e521b0b0	14.2	288	F			
10:10.3			1e521b0b0	14.56	256.5	F			
10:10.4			1e521b0b0	14.87		F			
10:10.5			1e521b0b0	15.19		F			
10:10.6			1e521b0b0	15.68		F			
10:10.7			1e521b0b0	16.15		F			
10:10.8			1e521b0b0	16.7	93	F			32 city
10:10.9			1e521b0b0	16.99	69	F			32 city
10:11.0			1e521b0b0	17.55	49.5	F			32 city
10:11.1			1e521b0b0	18.14		F			32 city
10:11.2			1e521b0b0	18.7		F			32 city
10:11.3			1e521b0b0	18.86		F			32 city
10:11.4			1e521b0b0	19.58		F			32 city
10:11.5			1e521b0b0	20.21	-25.5	F			32 city
10:11.6			1e521b0b0	20.63	-36	F			32 city
10:11.7			1e521b0b0	21.02	-37.5	F			32 city



Connected Car Data

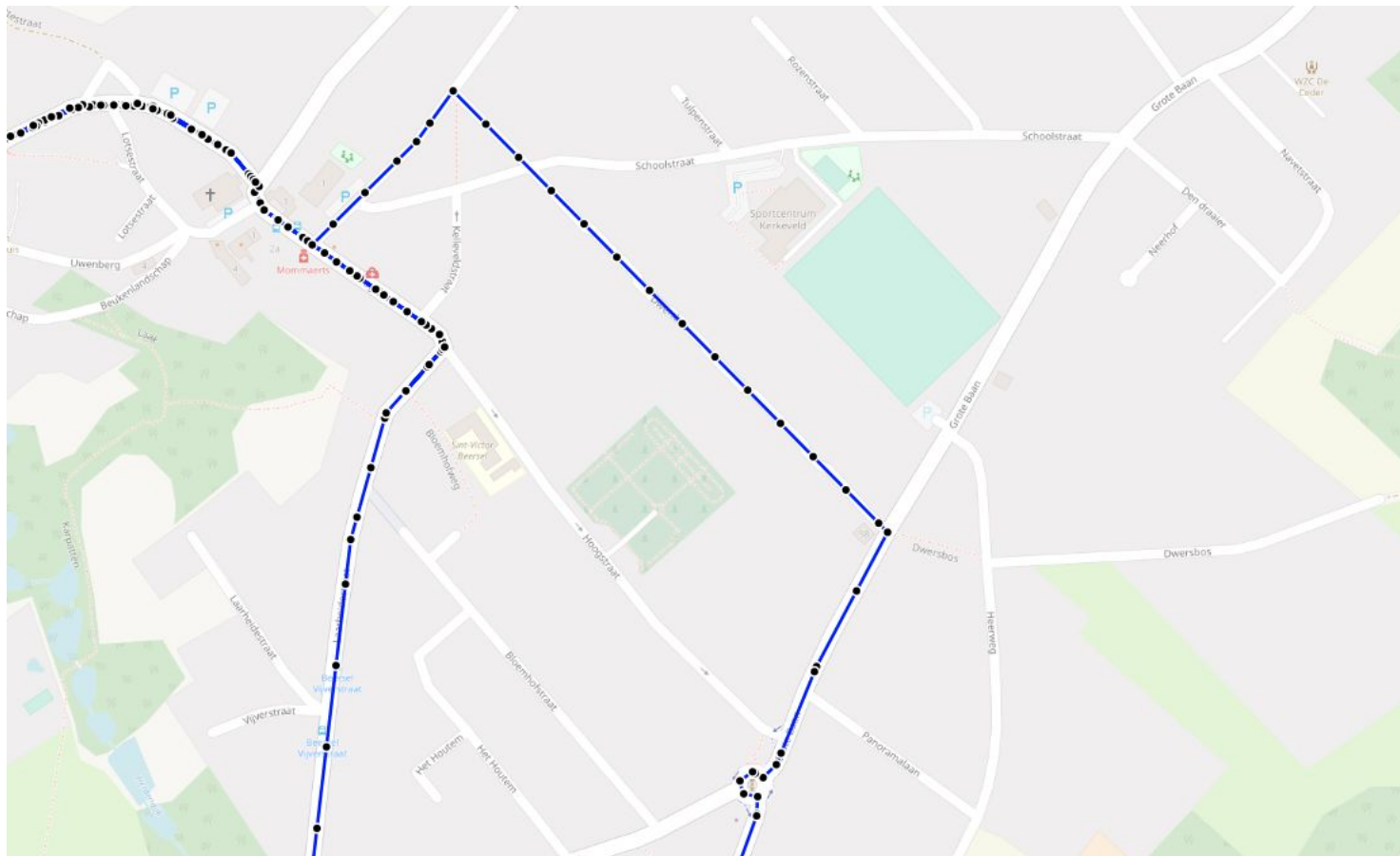


MobilityDB vs Time Series Databases

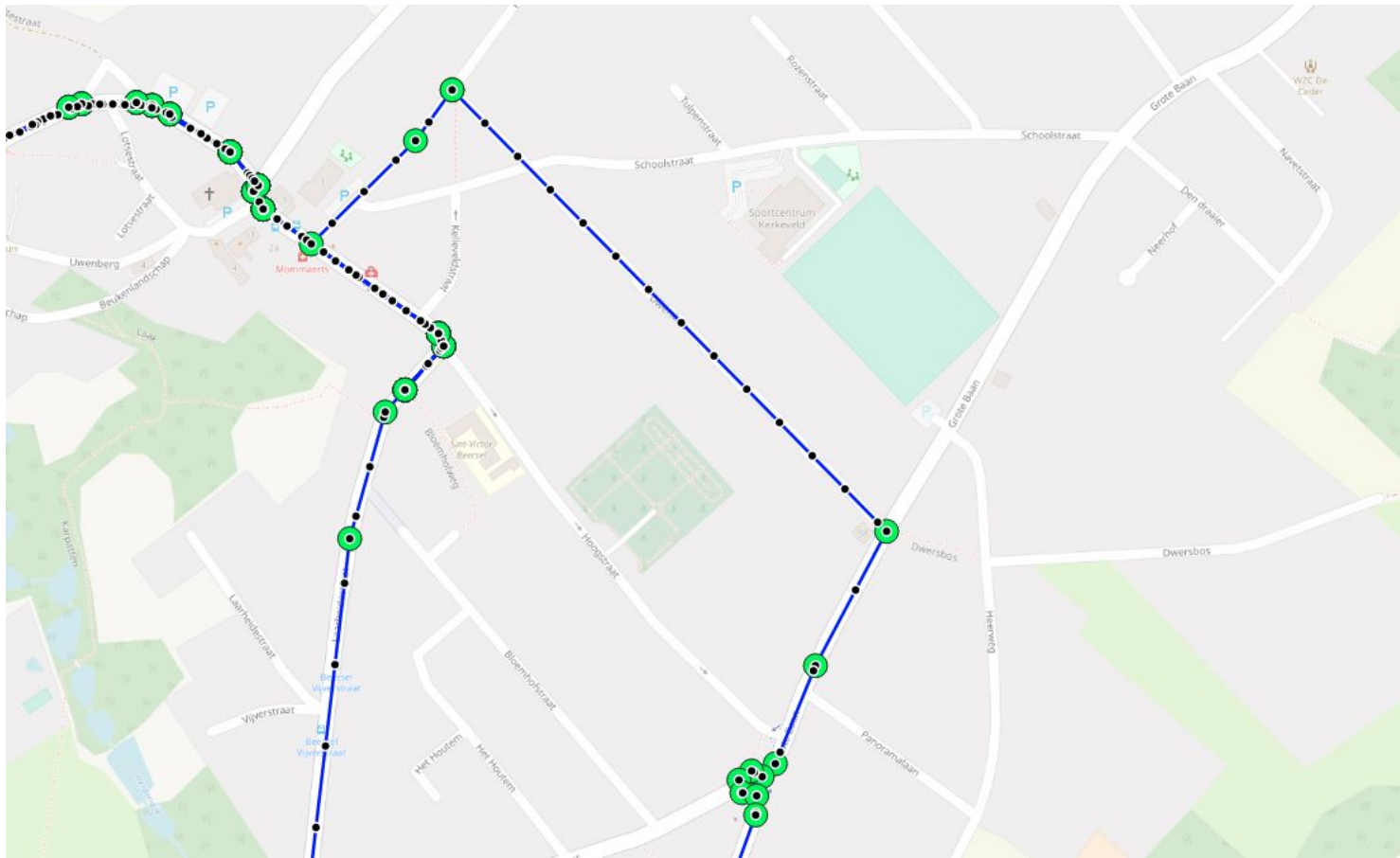


A detailed map of a residential and recreational area in Breda, The Netherlands. The map shows a network of streets including Schoolstraat, Groote Baan, and various smaller roads like Loozevliet and Knieveldstraat. A prominent route is marked with black dots, starting from the top left and curving through the center. Key landmarks include the 'Sportcentrum Knieveld' (a large green field), a church (marked with a cross), and several parks and green spaces. The map also shows a river (Loozevliet) and a canal (Knieveld). The area is characterized by a mix of urban development and natural greenery.

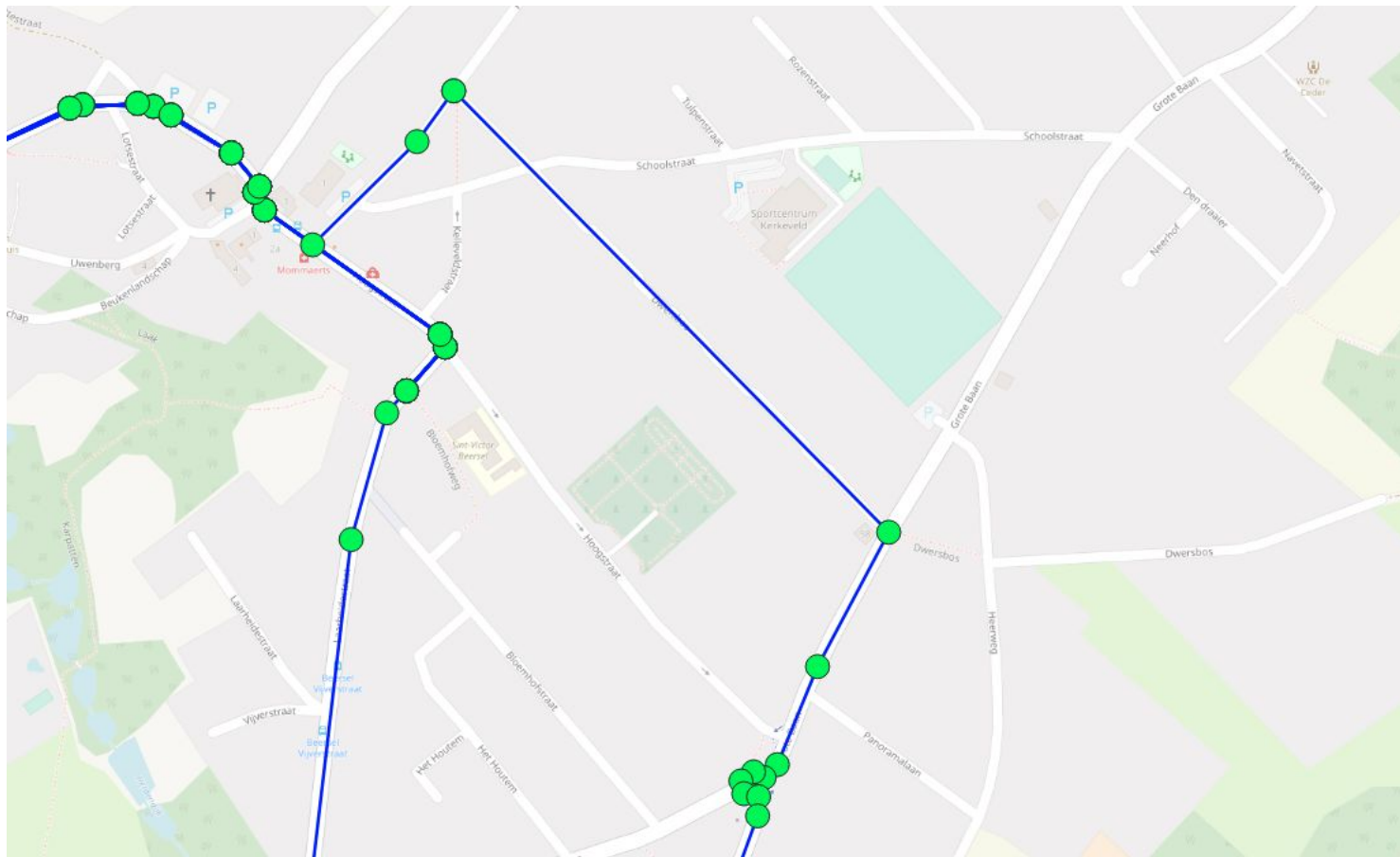
MobilityDB Compact Format



MobilityDB Compact Format



MobilityDB Compact Format



Migrating Connected Car Data into MobilityDB

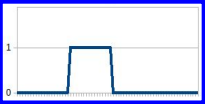
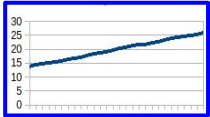
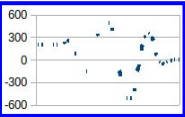
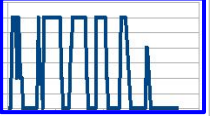
Compression rate 450%

timestamp	moving_state	trip_id	sp1	ssa	driver_gender	driver_age	speed_label
10:10.1		1 c521b0b0	13.64	309	F	32	city
10:10.2		1 c521b0b0	14.2	288	F	32	city
10:10.3		1 c521b0b0	14.56	256	F	32	city
10:10.4							city
10:10.5							city
10:10.6							city
10:10.7		1 c521b0b0	16.15		F	32	city
10:10.8		1 c521b0b0	16.7	93	F	32	city
10:10.9		1 c521b0b0	16.99	69	F	32	city
10:11.0		1 c521b0b0	17.55	49.5	F	32	city

16,000 records per trip

100 trip_id
1.6 million rows
165 MB



moving_state	trip_id	sp1	ssa	driver_gender	driver_age	speed_label
	c521b0b0			F	32	

100 trip_id
100 rows
37 MB

PostGIS -> MobilityDB migration

44384	2015-04-06 06:38:00	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:38:30	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:39:00	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:39:30	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:40:00	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:40:30	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:41:00	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:41:30	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:42:00	POINT(37.3826816 55.7937783)
44384	2015-04-06 06:42:30	POINT(37.3826816 55.7937783)



44384	[POINT(37.3826816 55.7937783)@2015-04-06 06:38:00+03, POINT(37.3826816 55.7937783)@2015-04-06 0...
44399	[POINT(37.6126166 55.7274032)@2015-04-06 07:14:29+03, POINT(37.6118683 55.7274732)@2015-04-06 0...
44399	[POINT(37.6127783 55.7265099)@2015-04-06 05:32:14+03, POINT(37.6127783 55.7265099)@2015-04-06 0...
62736	[POINT(37.6078283 55.7158566)@2015-04-06 05:35:17+03, POINT(37.607475 55.71504)@2015-04-06 05:35:...
62771	[POINT(37.6124233 55.7264416)@2015-04-06 05:07:57+03, POINT(37.6124233 55.7264416)@2015-04-06 0...
67756	[POINT(37.608135 55.7163933)@2015-04-06 04:47:23+03, POINT(37.60777 55.7153983)@2015-04-06 04:47:...
67762	[POINT(37.6093483 55.7190449)@2015-04-06 16:58:07+03, POINT(37.6094966 55.7188982)@2015-04-06 1...
67762	[POINT(37.6099266 55.7209099)@2015-04-06 04:41:30+03, POINT(37.60921 55.7190516)@2015-04-06 04:4...

10 billion rows a day

> 500 MB per day

~ 300 GB per year

15 thousand rows

~ 5 MB per day



2GB per year

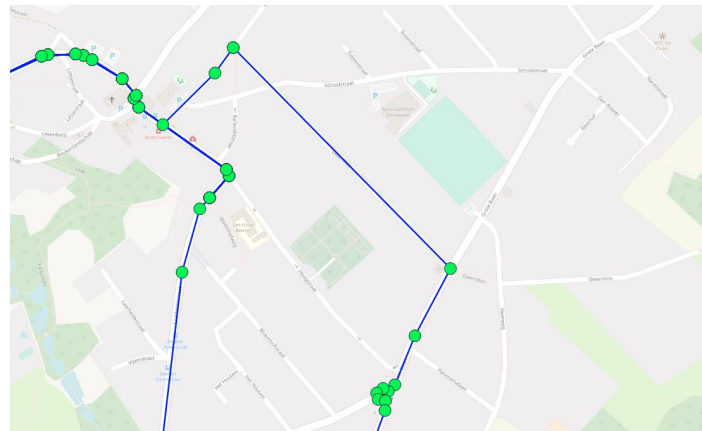
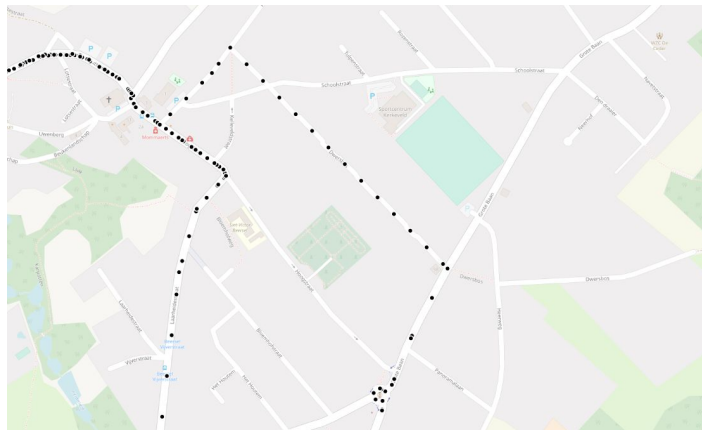
MobilityDB Compact Format

Improvement over existing tools

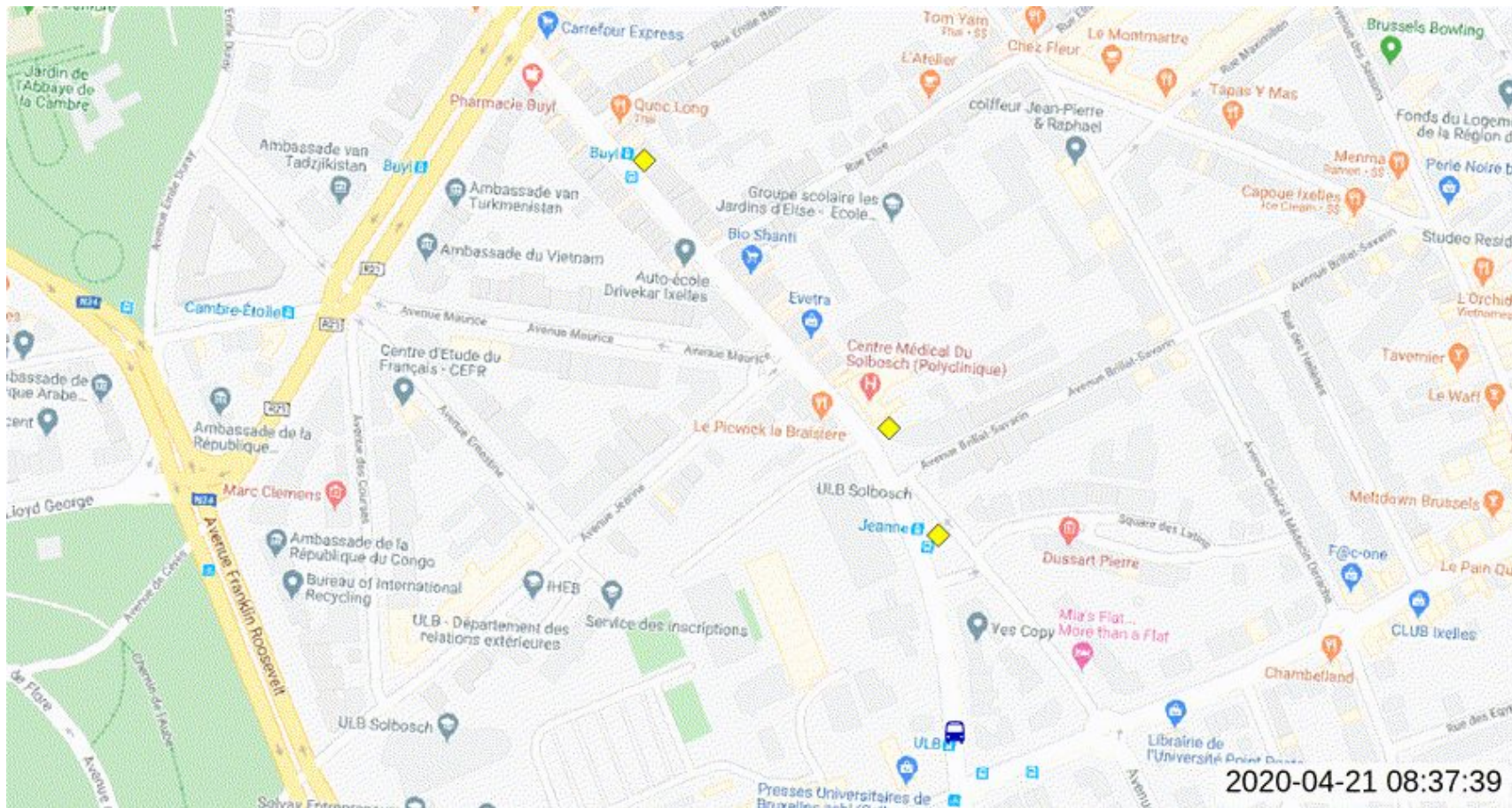
- Compact data storage with lossless high compression rate
- Query speedup
- Compatible with PostgreSQL ecosystem

Novel benefits

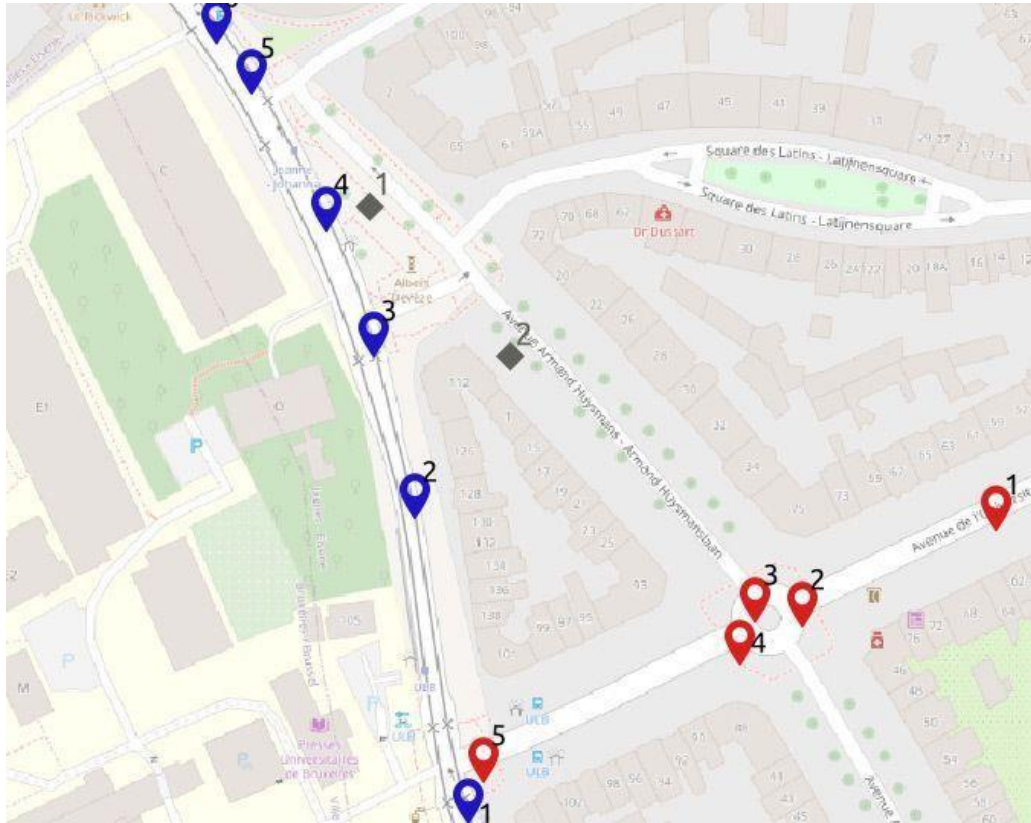
- Interpolation
- Native mobility data types
- Rich mobility analytics API
- Significant reduction in development time



Spatiotemporal Proximity: Smart Advertising



Spatiotemporal Proximity: Smart Advertising



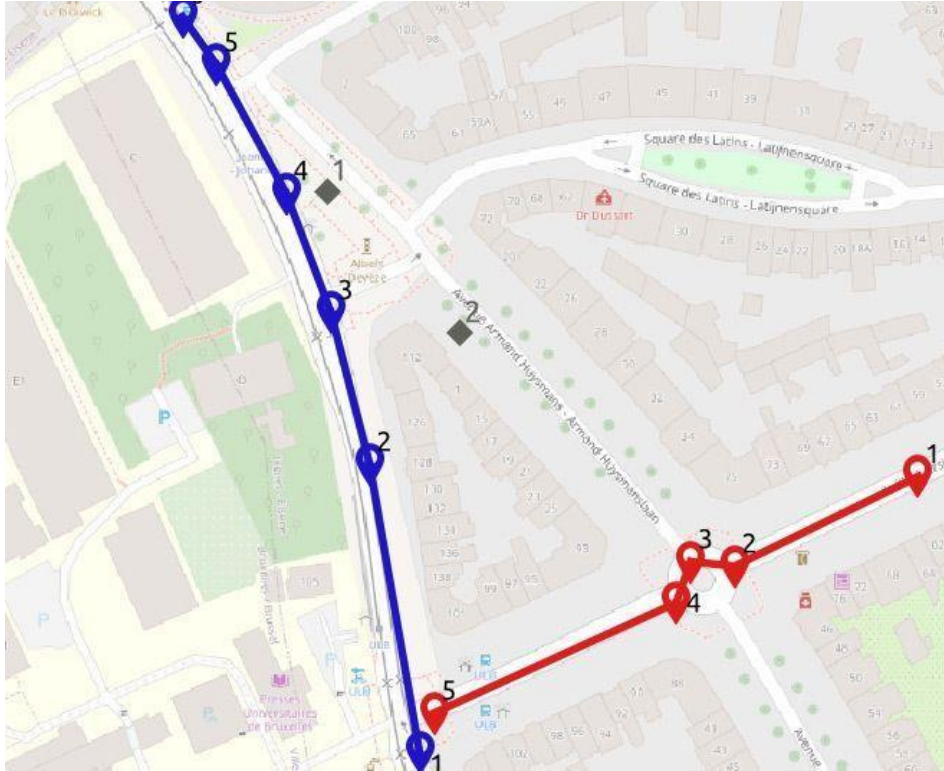
<https://techcommunity.microsoft.com/t5/azure-database-for-postgresql/analyzing-gps-trajectories-at-scale-with-postgres-mobilitydb-amp/ba-p/1859278>

```

WITH PointPair AS (
    SELECT Tripld, PointId AS P1, T AS T1, Geom AS Geom1,
        LEAD(PointId, 1) OVER (PARTITION BY Tripld ORDER BY PointId) AS P2,
        LEAD(T, 1) OVER (PARTITION BY Tripld ORDER BY PointId) AS T2,
        LEAD(Geom, 1) OVER (PARTITION BY Tripld ORDER BY PointId) AS Geom2
    FROM GpsPoint ),
Segment AS (
    SELECT Tripld, P1, P2, T1, T2, ST_MakeLine(Geom1, Geom2) AS Geom
    FROM PointPair WHERE P2 IS NOT NULL ),
Approach AS (
    SELECT Tripld, P1, P2, T1, T2, a.Geom, ST_Intersection(
        a.Geom, ST_ExteriorRing(ST_Buffer(b.Geom, 30))) AS VisibilityTogglePoint
    FROM Segment a, Billboard b WHERE ST_Dwithin(a.Geom, b.Geom, 30) )
SELECT Tripld, P1, P2, T1, T2, Geom, VisibilityTogglePoint,
(ST_LineLocatePoint(Geom, VisibilityTogglePoint) * (t2 - t1)) + t1 VisibilityToggleTime
FROM Approach;

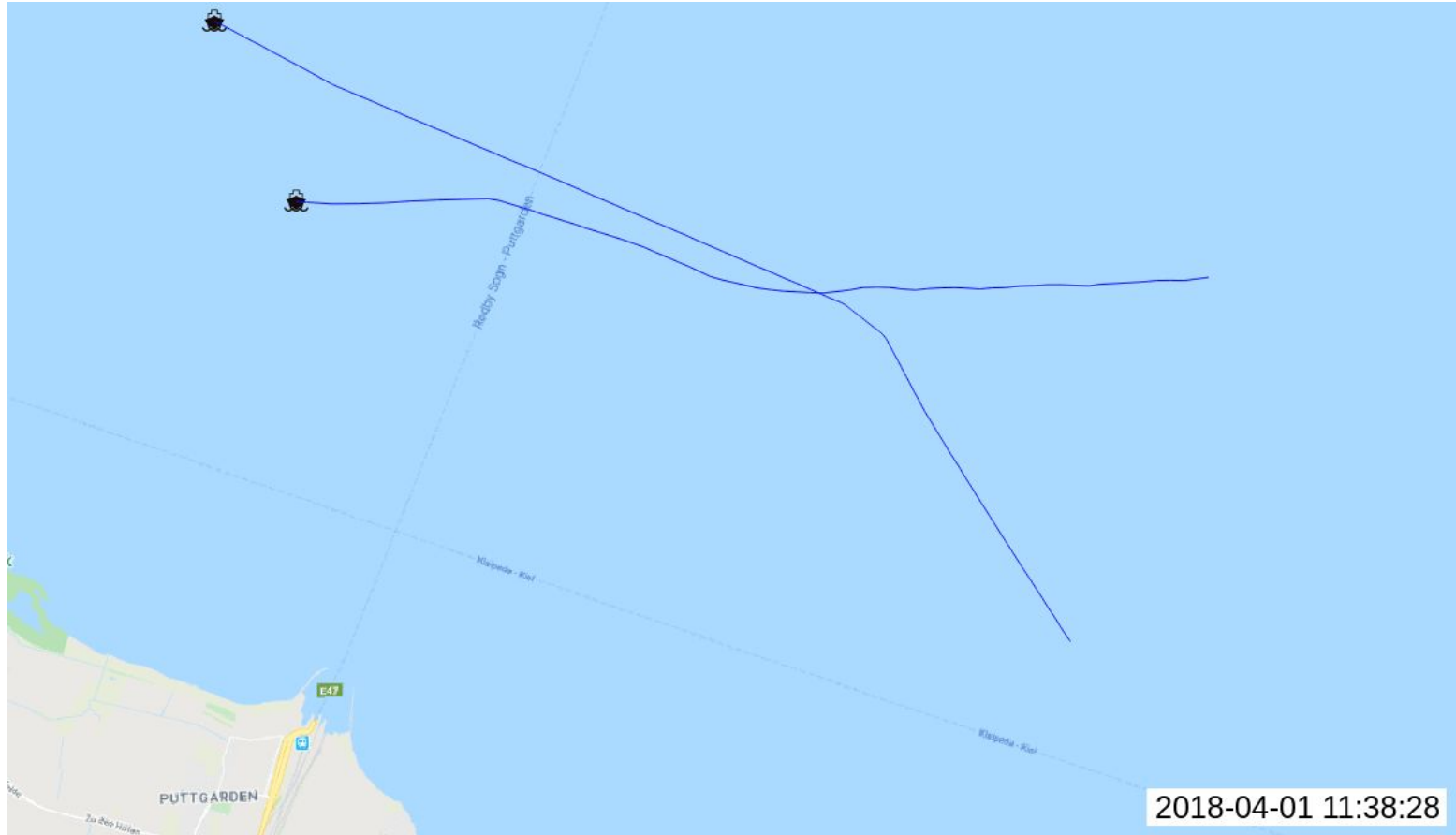
```


MobilityDB: Connecting the Dots

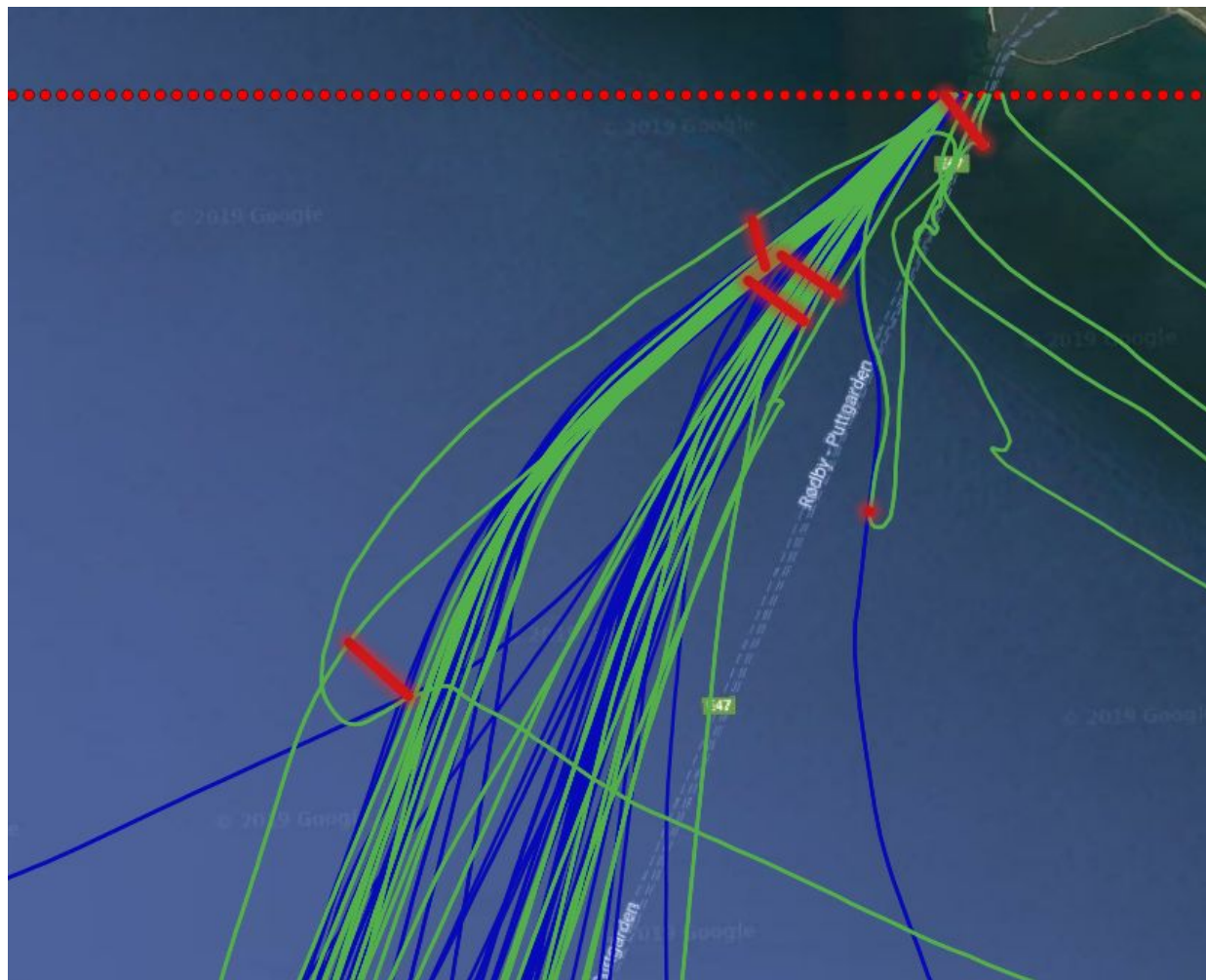


```
SELECT TripId, PointId, BillboardId  
FROM GpsPoint a, Billboard b  
WHERE ST_Dwithin(a.Geom,b.Geom,30);
```

Spatiotemporal Proximity: Marine Data







MobilityDB: Open Source based on OGC Standards

☰ README.md ✎

🔗 Main Build passing coverage 97%

🔄 code quality A chat on gitter

MobilityDB

An open source geospatial trajectory data management & analysis platform



<https://github.com/MobilityDB/MobilityDB>

OSGeo ☰

Home » Projects » MobilityDB

MobilityDB

An open source geospatial trajectory data management & analysis platform

◀ Back to projects

<https://www.osgeo.org/projects/mobilitydb/>

Open Geospatial Consortium ☰ Menu

Moving Features SWG

Chair(s):
Ishimaru, Nobuhiro (Ishimaru, Nobuhiro) - Co-Chair,
Kim, Kyoung-Sook (National Institute of Advanced Industrial Science & Technology (AIST)) - Co-Chair,
SAKR, Mahmoud (Université Libre de Bruxelles (ULB)) - Co-Chair

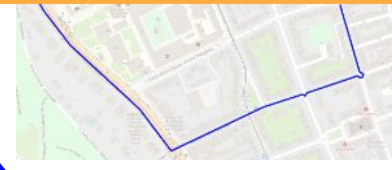
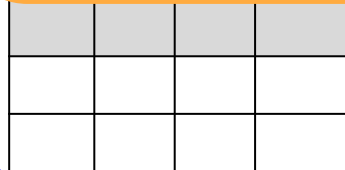
Group Charter:
[Download Charter document](#)

Group Description:
Moving Features
With the development of communication and positioning technology such as Global Navigation Satellite System (GNSS), Wi-Fi, and Beacon, collecting movement data for moving features,

<https://www.ogc.org/projects/groups/movfeatswg>



SQL Optimization	Scalar statistics & selectivity estimation	Grid-based statistics	Spatial grid + period bound histograms
Indexes	B-tree, hash, GiST, SP-GiST, GIN, BRIN	GiST, SP-GiST, BRIN	GiST, SP-GiST
Operations	Comparison, transformation, casting, ...etc	topological, CRS, properties, overlay, ...etc.	trajectory, temporal properties, lifted predicates, aggregations
Type System	numeric, character, date/time, bool, xml, json	Geometry, geography	tgeompoint, tgeogpoint, tint, tfloat, ttext, tbool



Project[About](#)[Licence](#)**Documentation**[Data Model](#)[Normalization](#)[Data Structures](#)[Aggregate Operations](#)[Developer's Documentation](#)**Moving Features Formats**[Well-Known Text \(WKT\)](#)[Well-Known Binary \(WKB\)](#)[Moving-Features JSON \(MF-JSON\)](#)**Tutorial Programs**[My First MEOS Program](#)[Read from File](#)[Assemble Trips](#)[Store in MobilityDB](#)[Disassemble Trips](#)[Clip Trips to Geometries](#)[Tile Trips](#)

MEOS

MEOS (Mobility Engine, Open Source) is a C library and its associated API for manipulating temporal and spatiotemporal data. It is the core component of [MobilityDB](#), an open source geospatial trajectory data management & analysis platform built on top of [PostgreSQL](#) and [PostGIS](#).

MEOS extends the [ISO 19141:2008](#) standard (Geographic information — Schema for moving features) for representing the change of non-spatial attributes of features. It also takes into account the fact that when collecting mobility data it is necessary to represent “temporal gaps”, that is, when for some period of time no observations were collected due, for instance, to signal loss.

MEOS is heavily inspired by a similar library called [GEOS](#) (Geometry Engine, Open Source) — hence the name. A [first version](#) of the MEOS library written in C++ has been proposed by Krishna Chaitanya Bommakanti. However, due to the fact that MEOS codebase is actually a subset of MobilityDB codebase, which is written in C and in SQL, the current version of the library allows us to evolve both [programming environments](#) simultaneously.

MEOS aims to be the base library on which other projects can be built. For example, the following projects are built on top of MEOS:

- [PyMEOS](#) is a Python binding to MEOS using [CFFI](#)
- [MobilityDB](#) is a PostgreSQL extension that enables storing and manipulating the temporal types provided by MEOS.

Other projects can be built on top of MEOS, for example, Java or C# drivers for MEOS or implementing MEOS on other DBMSs such as MySQL.

Project

[Support](#)
[Code of Conduct](#)
[Project Steering Committee](#)

Development

[CI Status](#)
[Requests for Comment](#)
[Testing](#)

Usage

[Download and Build](#)
[Install Packages](#)
[API Docs](#)
[C API Programming](#)
[C++ API Programming](#)
[Tools](#)
[Bindings](#)
[FAQ](#)

Geometry Formats

[GeoJSON](#)
[Well-Known Binary \(WKB\)](#)
[Well-Known Text \(WKT\)](#)

News

[Version 3.10.0](#)

GEOS

GEOS is a C/C++ library for [computational geometry](#) with a focus on algorithms used in [geographic information systems](#) (GIS) software. It implements the [OGC Simple Features](#) geometry model and provides all the spatial functions in that standard as well as many others. GEOS is a core dependency of [PostGIS](#), [QGIS](#), [GDAL](#), and [Shapely](#).

If you need support using the GEOS library or would like to get involved in the community check out the [Support](#) page.

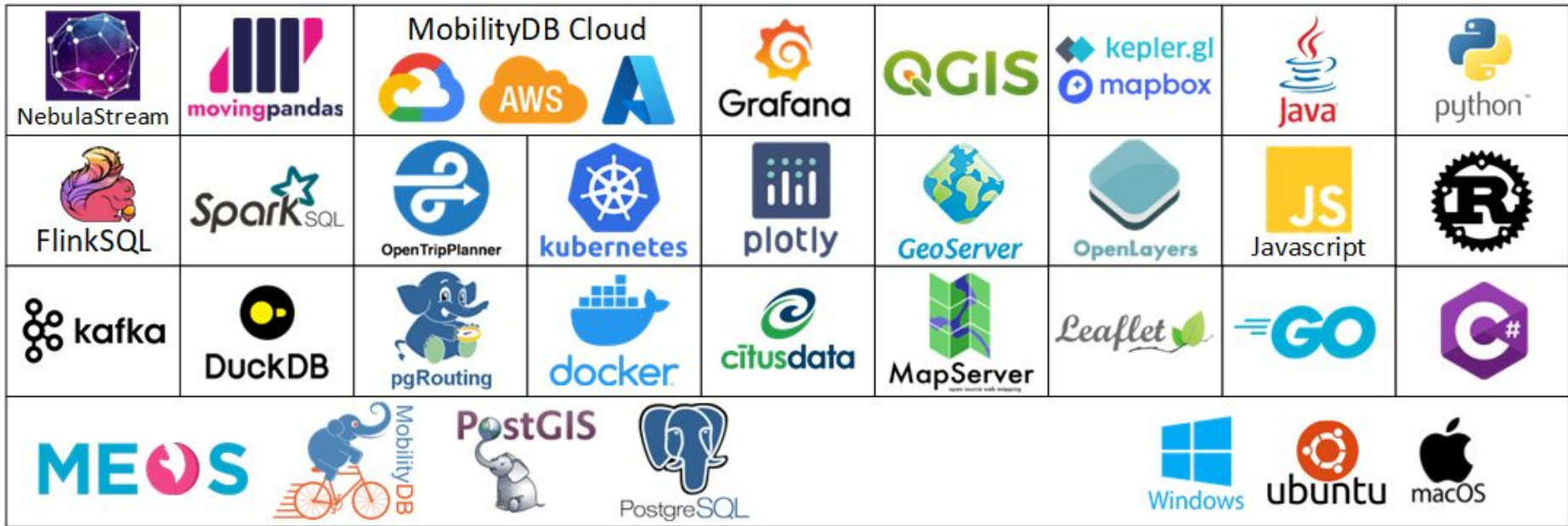
Capabilities

Spatial Model and Functions

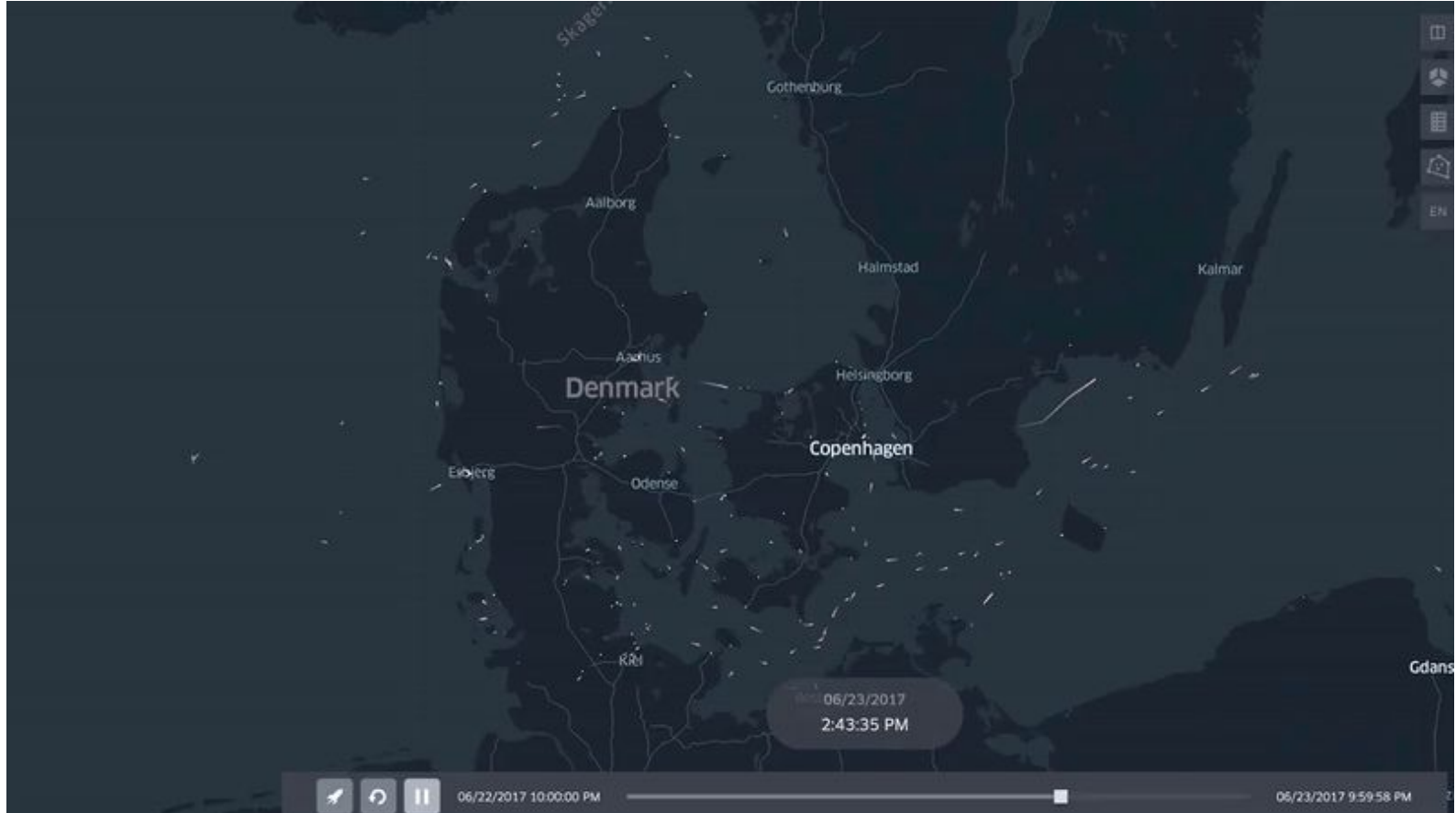
- **Geometry Model:** Point, LineString, Polygon, MultiPoint, MultiLineString, MultiPolygon, GeometryCollection
- **Predicates:** Intersects, Touches, Disjoint, Crosses, Within, Contains, Overlaps, Equals, Covers
- **Operations:** Union, Distance, Intersection, Symmetric Difference, Convex Hull, Envelope, Buffer, Simplify, Polygon Assembly, Valid, Area, Length,
- **Prepared geometry** (using internal spatial indexes)
- **Spatial Indexes:** STR (Sort-Tile-Recursive) packed R-tree spatial index
- **Input/Output:** OGC Well Known Text (WKT) and Well Known Binary (WKB) readers and writers.

API Features

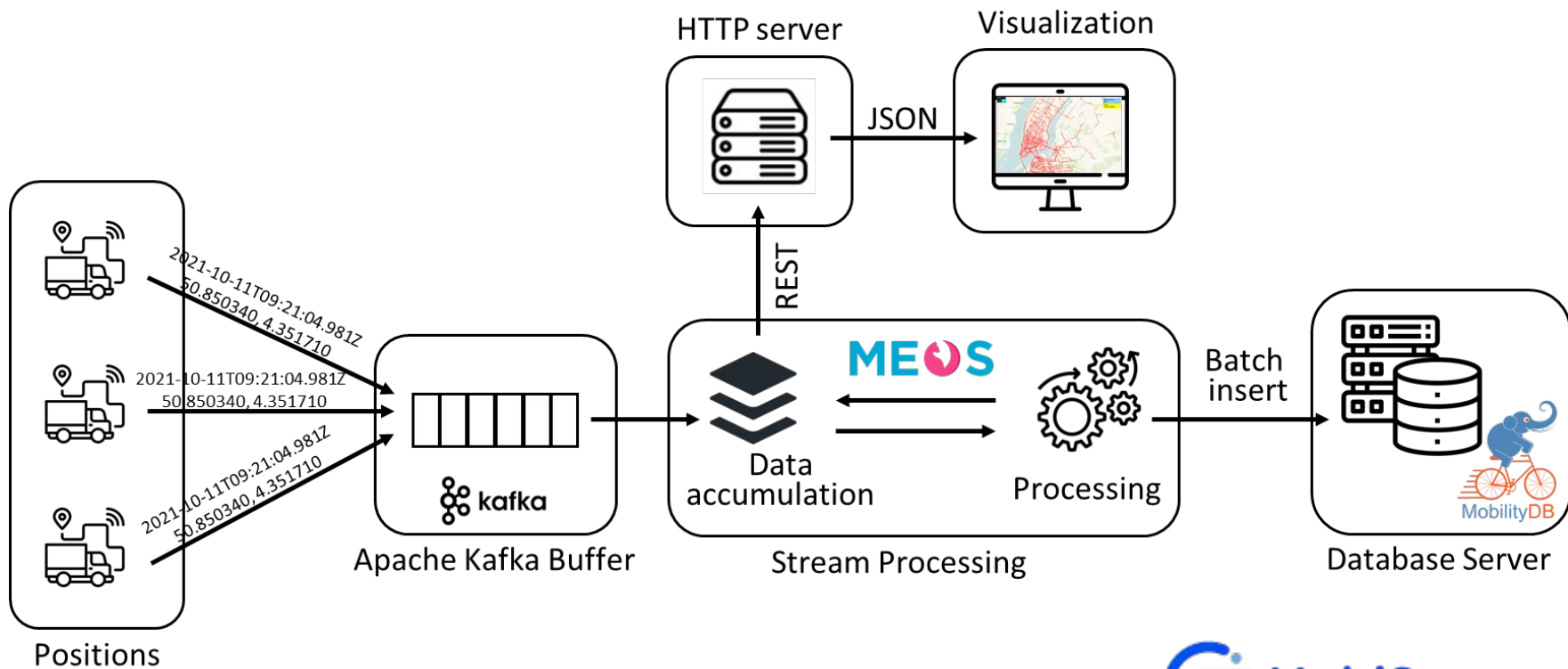
MobilityDB Ecosystem



Visualizing Mobility Data in Deck.gl



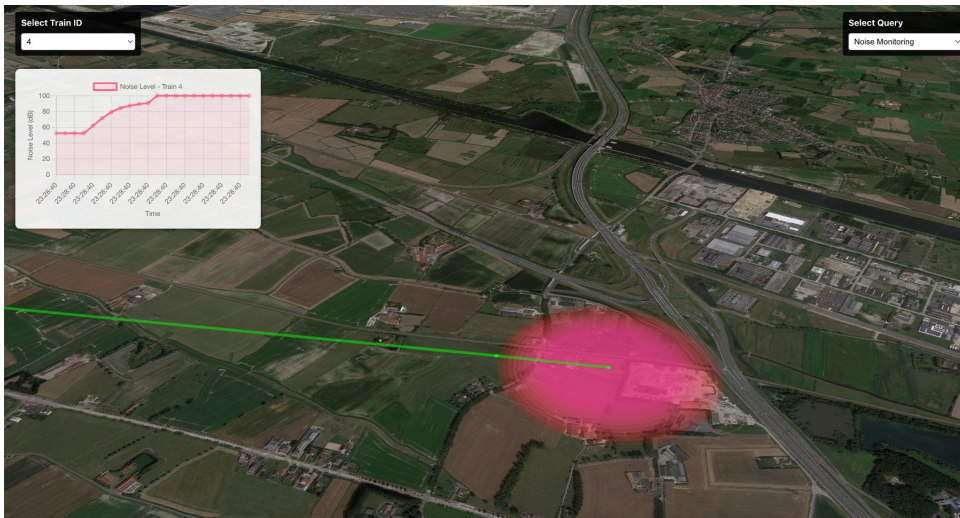
Mobility Data in Stream and Edge Processing



Geofencing Queries in MobilityNebula

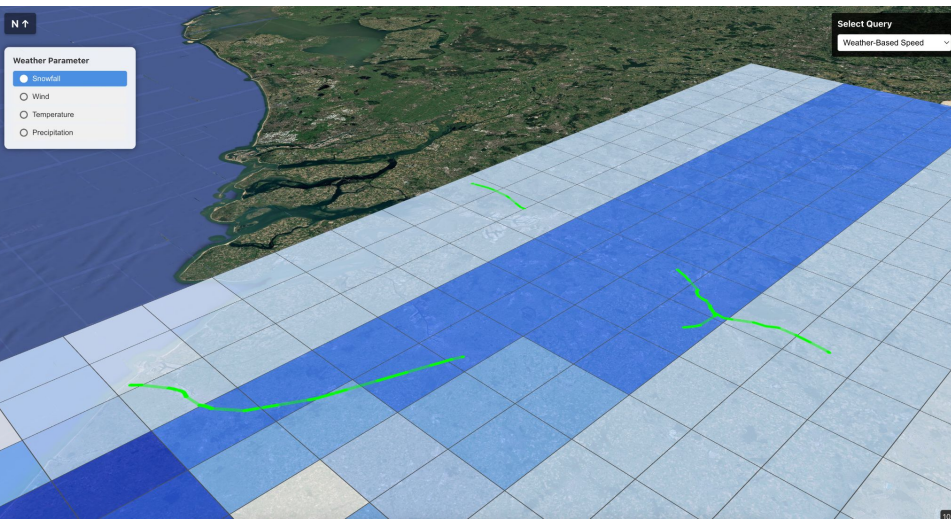


Location-Based Alert Filtering

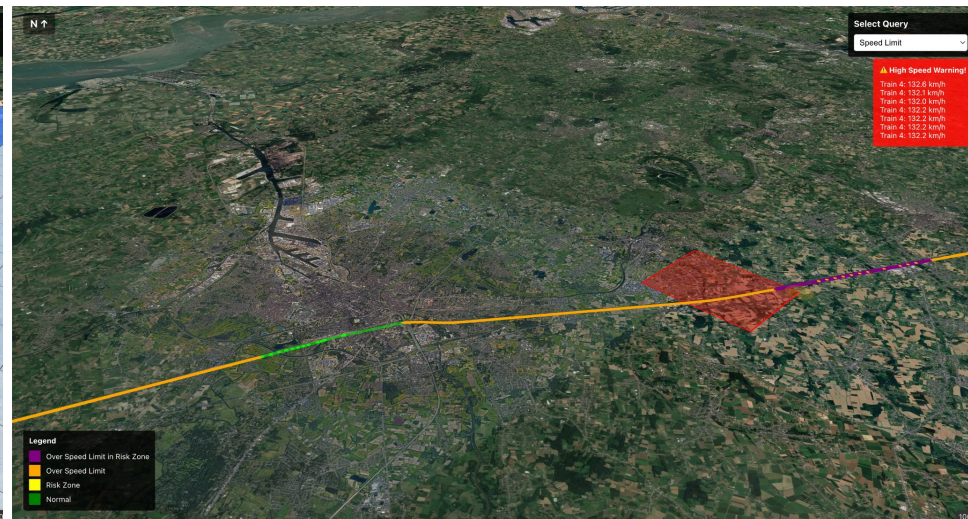


Location-Based Noise Monitoring

Geofencing Queries in MobilityNebula



 Weather-Based Speed Zones



 Dynamic Speed Limit

Geofencing Queries in MobilityNebula

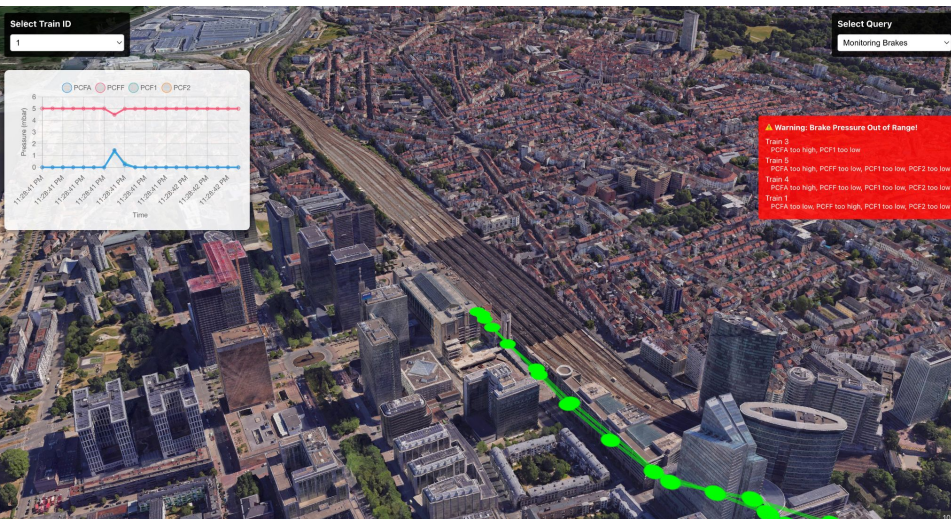


Passenger load estimation

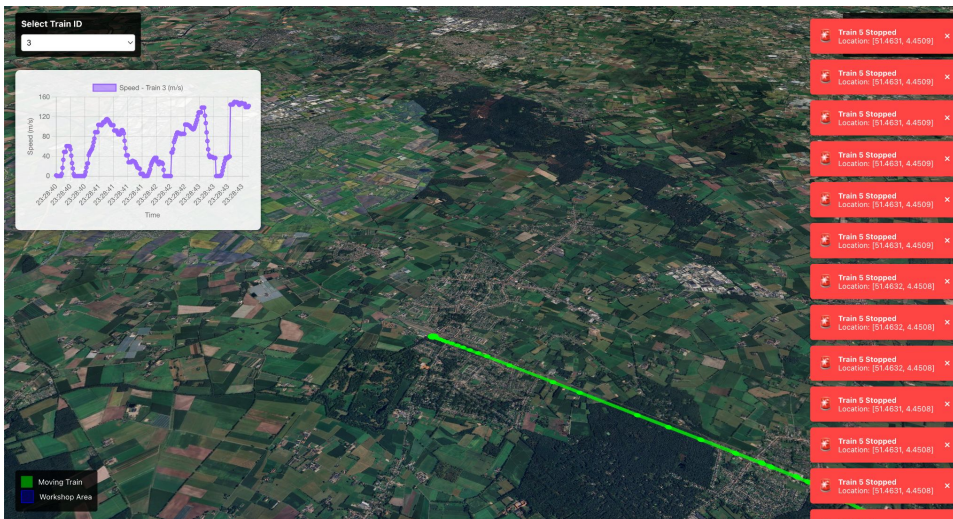


Battery monitoring and nearby workshop detection

Geofencing Queries in MobilityNebula

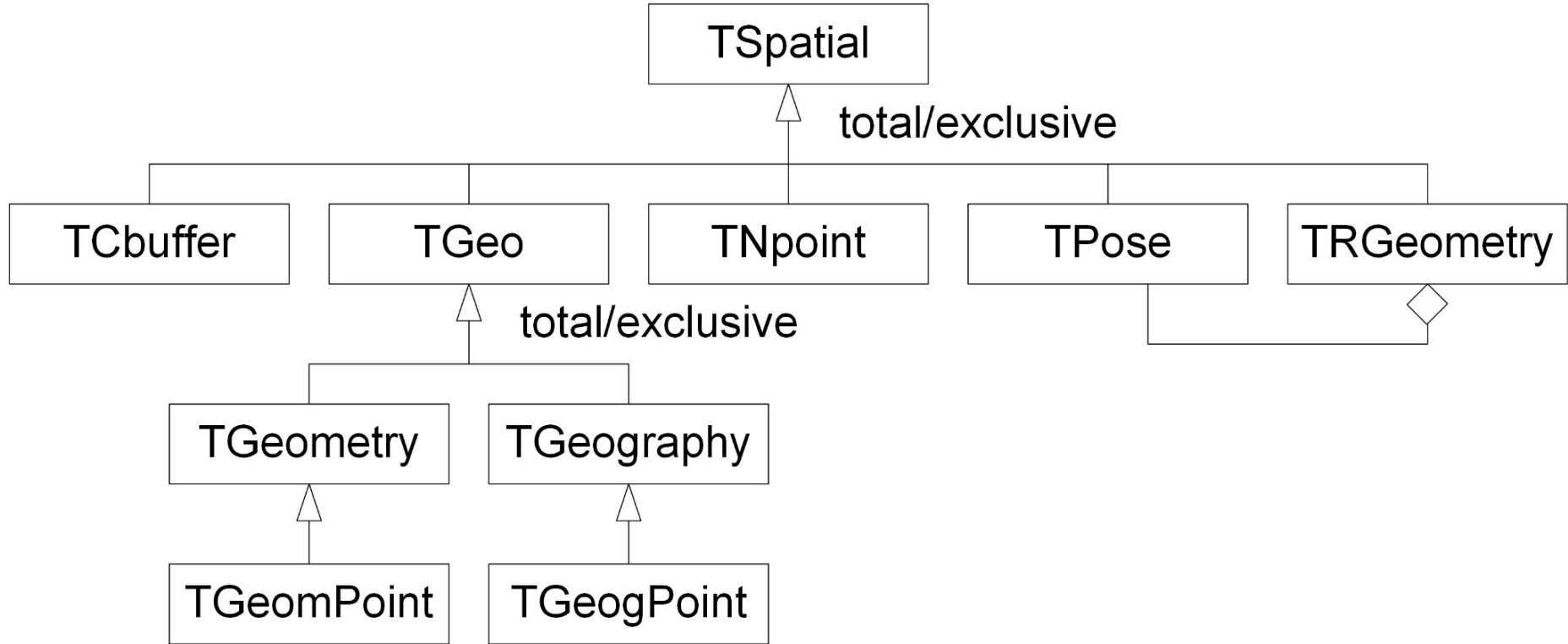


🔲 Brake monitoring

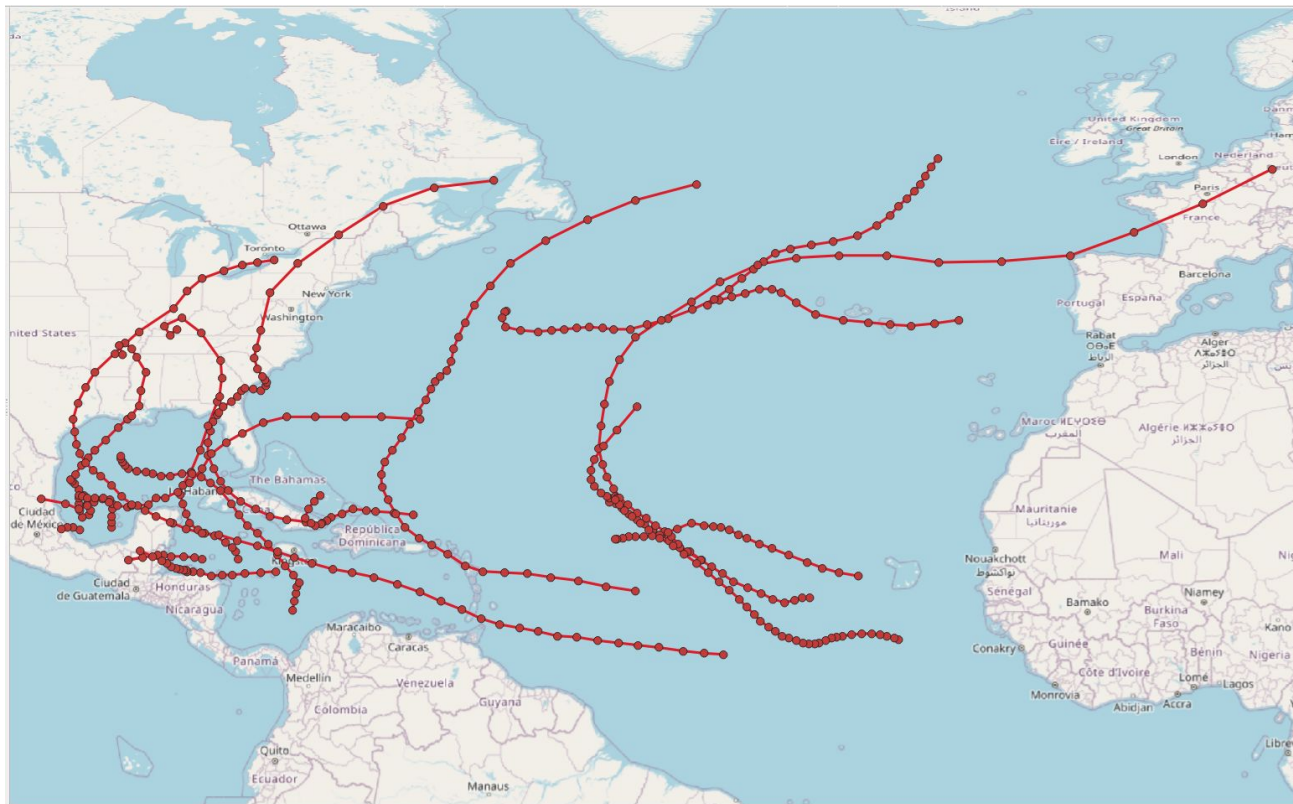


🔲 Detection of unscheduled stops

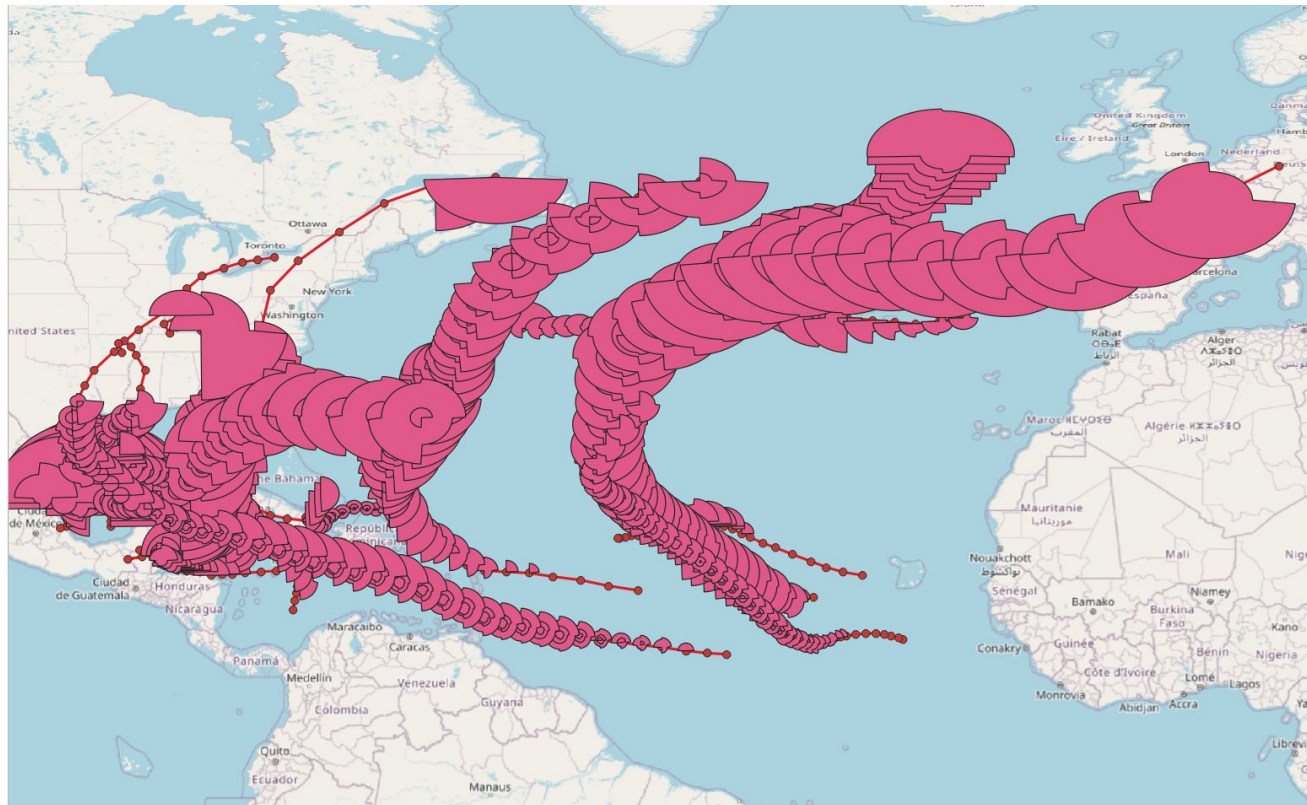
Current Developments: New Spatiotemporal Types



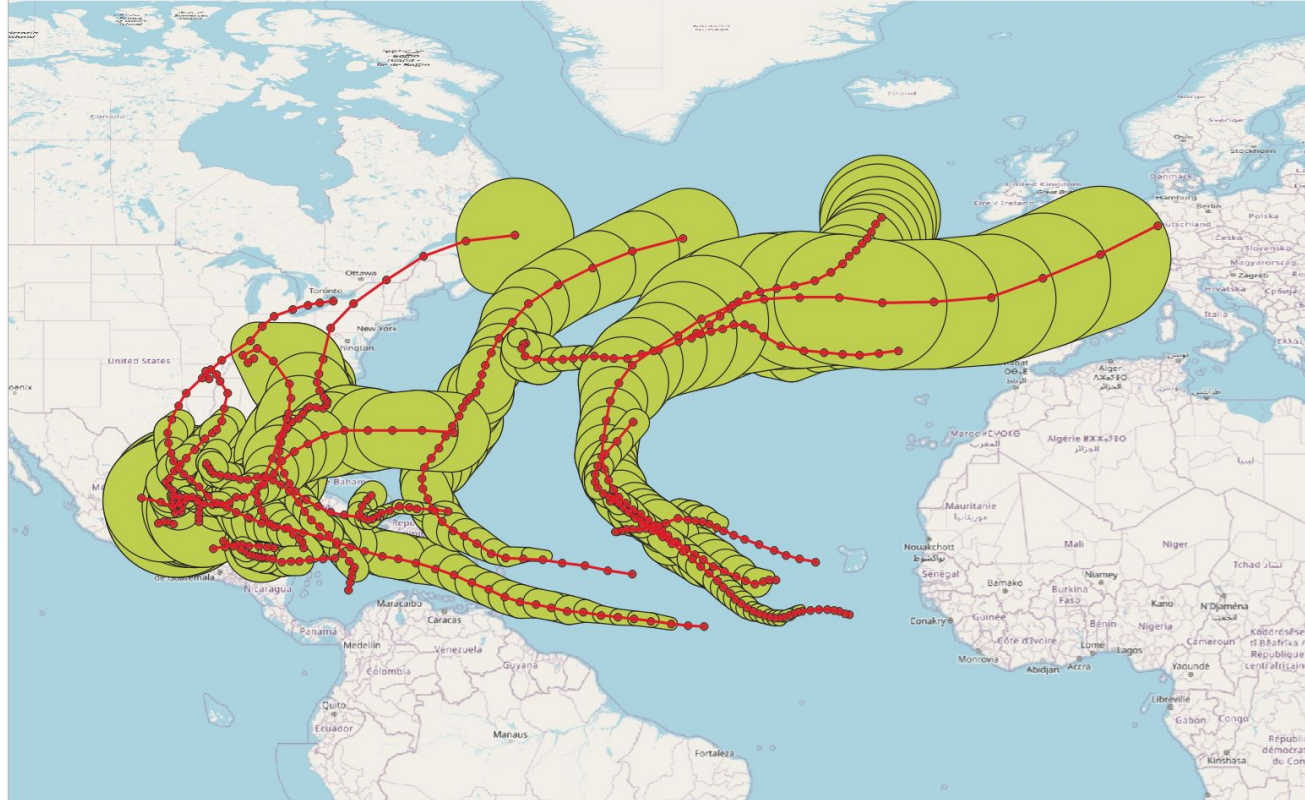
Temporal Points



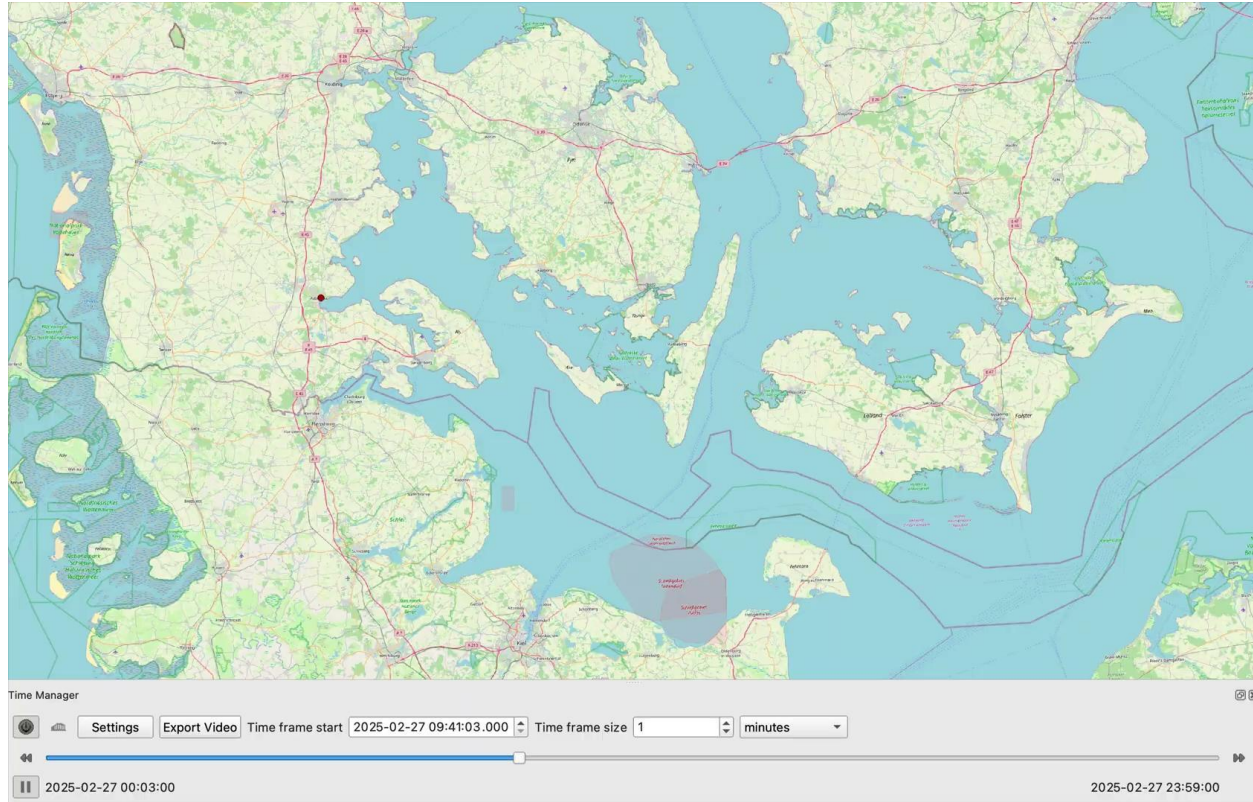
Temporal Geometries



Temporal Circular Buffers



Temporal Circular Buffers for Maritime Data



MobilityDB

An open source geospatial trajectory data management & analysis platform.

 [View on Github](#)

Many thanks for your attention !

Location tracking devices, such as GPS, are nowadays widely used in smartphones, and in vehicles. As a result, geospatial trajectory data are currently being collected and used in many application domains. MobilityDB provides the necessary database support for storing and querying such geospatial trajectory data

MobilityDB is implemented as an extension to PostgreSQL and PostGIS. It implements persistent database types, and query operations for managing geospatial trajectories and their time-varying properties.



[Learn More](#)

[OPEN CHAT](#)